CPSC 335, Algorithm Engineering – Spring 2026
Project 1: Implementing Algorithms

Student Learning Objectives:

Demonstrate your ability to

- Translate descriptions of algorithms into pseudocode
- Analyze your pseudocode mathematically
- Implement your algorithm in C++
- Test your implementation
- Describe your results
- Observe timings correspond to Big-O trends
- Translate requirements into solutions in a timely manner

Description - The Alternating Disk Problem:

The problem below is slightly changed from the one presented in Levitin's textbook as Ex. 14 on page 103.

You have a row of 2n disks of two colors, n light and n dark. They alternate: light, dark, light, dark, and so on. You want to get all the light disks to the left-hand end, and all the dark disks to the right-hand end. The only moves you are allowed to make are those that interchange the positions of two adjacent disks (i.e. they touch each other). No other moves are allowed. Design an algorithm for solving this puzzle and determine the number of moves it takes.

The *alternating disks problem* is:

Input:

1. a positive integer n, and
2. a list of 2n disks of alternating light and dark disks, always starting with light. For example:

Output**:**

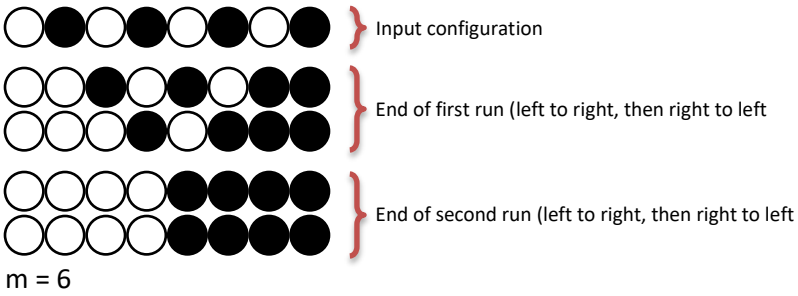
1. a non-negative integer m representing the number of interchanges (swaps) performed to arrange all the dark ones after all the light ones.
2. a list of 2n disks, the first n disks are light, the next n disks are dark. For example:

There are two algorithms presented below that solve this problem in $O(n^2)$ time. An improvement can be obtained by not going all the way to the left or to the right, since some disks at the ends are already in the correct position. You need to translate the descriptions of the two algorithms into clear pseudocode. You are allowed to make improvements as long as they don't change the description of the algorithm.

Algorithm 1: The lawnmower algorithm

It starts with the leftmost disk and proceeds to the right one at a time until it reaches the rightmost disk: compares adjacent disks (i.e., $disk_k$ with $disk_{k+1}$) and swaps them only if necessary. Now we have two light disks at the left-hand end and two dark disks at the right-hand end. Once it reaches the right-hand end, it starts with the rightmost disk, compares adjacent disks(i.e., $disk_k$ with $disk_{k-1}$) and proceeds to the left until it reaches the leftmost disk, doing the swaps only if necessary. The lawnmower movement is repeated $\lceil n/2 \rceil$ times.

Consider the example below when n=4. The first row is the input configuration, the second row is the end of comparison from left to right, the third row is the end of the first run (round trip that contains left to right followed by right to left), etc.

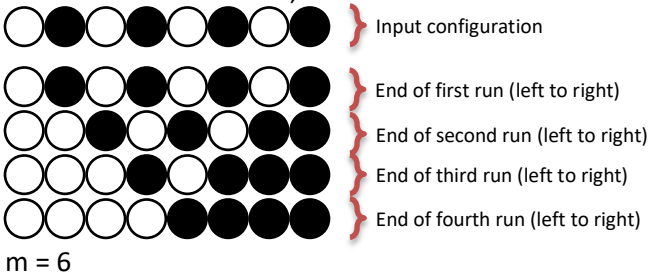


Algorithm 2: The alternate algorithm

It starts with the leftmost disk and proceeds to the right until it reaches the rightmost disk: compares adjacent disks and swaps them only if necessary. It does not, however, iterate through each index, but instead iterates over each ***pair*** of disks (i.e., it moves 2 steps at a time). This time, we consider a run to be a check of adjacent disks only from left-to-right.

Next it starts with the second leftmost disk and proceeds to the right until it reaches the second rightmost disk: compares adjacent disks and swaps them only if necessary. This is the end of Run 2. Now we have two light disks at the left-hand end and two dark disks at the right-hand end. Next it is Run 3 that proceeds exactly as Run 1, starting with the leftmost disk. Run 3 is followed by Run 4 that is exactly as Run 2, starting with the second leftmost disk. So really, Run 1 and Run 2 continually alternate until sorting has finished. There are a total of n runs.

Consider the example below when n=4. The first row is the input configuration, the second row is the end of run 1, the third row is the end of run 2, etc.



Algorithm Design

Your first task is to design an algorithm for each of the two problems. Write clear pseudocode for each algorithm. This is not intended to be difficult; the algorithms I have in mind are all relatively simple, involving only familiar string operations and loops or nested loops. Do not worry about making these algorithms exceptionally fast; the purpose of this experiment is to see whether observed timings correspond to Big-O trends, not to design impressive algorithms.

Mathematical Analysis

Your next task is to analyze each of your two algorithms mathematically. You should prove a specific Big-O efficiency class for each algorithm. These analyses should be routine, similar to the ones we have done in class and in the textbook. I expect each algorithm's efficiency class will be one of $O(n)$, $O(n^2)$, $O(n^3)$, or $O(n^4)$.

What to Do

1. **Project Report:** Produce and deliver a brief **handwritten** Project Report scanned into ***PDF format***. Your handwriting must be absolutely and perfectly legible to earn credit. Treat this not as work in progress, but rather your final, polished, college-level quality delivery to your customer. Your report should include the following:
   a. Algorithm 1: The lawnmower algorithm
      i. Python-like pseudocode
      ii. Compute the step count
         1. Provide your Time Estimation function *T(n) = …*
         2. Evaluate your function for 3 unique values for n showing the resulting step count
      iii. A brief proof argument for the time complexity efficiency class
   b. Algorithm 2: The alternate algorithm
      i. Python-like pseudocode
      ii. Compute the step count
         1. Provide your Time Estimation function *T(n) = …*
         2. Evaluate your function for 3 unique values for n showing the resulting step count
      iii. A brief proof argument for the time complexity efficiency class
2. **Project Programming:** Implement, test, and deliver your algorithm implementation project starting with the Student Files provided.

Rules and Constraints:

1. You are to modify only designated TO-DO sections using only the techniques discussed in class. The grading process will detect and discard any changes made outside the designated TO-DO sections, including spacing and formatting. Designated TO-DO sections are identified with the following comments:

```
////////////////////////// TO-DO (X) ////////////////////////////////
...
////////////////////// END-TO-DO (X) ////////////////////////////////
```

   Keep and do not alter these comments. Insert your code between them.

2. You may work in groups of up to 2 people, but you must deliver individually. The Project Report must be individually handwritten by each group member (No, you can't deliver your groupmates handwritten report as your own.)

Reminders:

- Multiple submissions may be available and a waiting period between submissions may be enforced.
- The C++ using directive `using namespace std;` is **never allowed** in any file in any deliverable product. Being new to C++, you may have used this in the past. If you haven't done so already, it's now time to shed this crutch and fully decorate your identifiers.
- A clean compile is an entrance criterion. Deliveries that do meet the entrance criteria cannot be graded.
- The grading process uses Build.sh on Ubuntu to compile and link your program. There is nothing magic about Build.sh, all it does is save you (and me) from repeatedly typing the very long compile command and all the source files to compile.
- Filenames are case sensitive, both in source code and in your OS file system. Windows doesn't care about filename case, but Linux does.
- You may redirect standard input from a text file, and you must redirect standard output to a text file named output.txt. Failure to include output.txt in your delivery indicates you were not able to execute your program and will be scored accordingly. A screenshot of your terminal window is not acceptable. See How to build and run your programs. Also see How to use command redirection under Linux if you are unfamiliar with command line redirection.

Deliverable Artifacts:

Provided files	Files to deliver	Comments
Algorithm.hpp	1. Algorithm.hpp	Start with the file provided. Make changes in the designated TO-DO sections (only). The grading process will detect and discard all other changes.
Disk.hpp Disks.hpp main.cpp	2. Disk.hpp 3. Disks.hpp 4. main.cpp	You shall not modify these files. The grading process will overwrite whatever you deliver with the one provided with this assignment. It is important your delivery is complete, so don't omit these files.
	5. output.txt	Capture your program's output to this text file using command line redirection. See command redirection. Failure to deliver this file indicates you could not get your program to execute. Screenshots or terminal window log files are not permitted.
Report_Template.pdf	6. Project_1_Report.pdf	Print the template, add your name, CWID, and handwritten report content. Add additional pages if necessary. Scan your completed work and save as a .pdf file named as specified in the previous column.
	Readme.txt	Optional. Use it to communicate your thoughts to the grader.
sample_output.txt		A sample of a working program's output. Your output may vary.